

DirectX 12

이펙트 라이브러리 매뉴얼

통합 · 초기화 · 호출 · 확장 · 포스트 프로세싱

싱글톤 기반 이펙트 관리 / 오브젝트 풀링 / 파티클·메시 이펙트 /

로컬·리모트 부스터 / 사용자 정의 프리셋 / 방사형 블러·스피드 라인

빠른 목차

1. 구성 파일과 프로젝트 설정
2. 초기화 및 종료
3. 프레임 업데이트·렌더링 순서
4. 단발성·이벤트 큐·이름 기반 호출
5. ParticleConfig와 사용자 정의 효과
6. 부스터·바람막이·로컬/리모트 제어
7. 특수 효과: 드리프트, Lock Orbit, Stun Orbit
8. 방사형 블러·스피드 라인 포스트 프로세싱
9. API 요약, 호환성 조건, 문제 해결

가장 중요한 순서 사전 설정 → Initialize → InitializePostProcess → 매 프레임 Update → Scene 렌더 → Effect Render → RenderRadialBlur → 종료 전 GPU 대기 → Release

1. 라이브러리 개요

CEffectLibrary는 DirectX 12 렌더링 프로젝트에 파티클·메시·포스트 프로세싱 이펙트를 추가하는 싱글톤 관리자입니다. 효과 인스턴스를 미리 생성해 두는 오브젝트 풀과 ActiveEffect 수명 관리를 사용하며, 호출 측은 EFFECT_TYPE 또는 등록 이름으로 효과를 재생합니다.

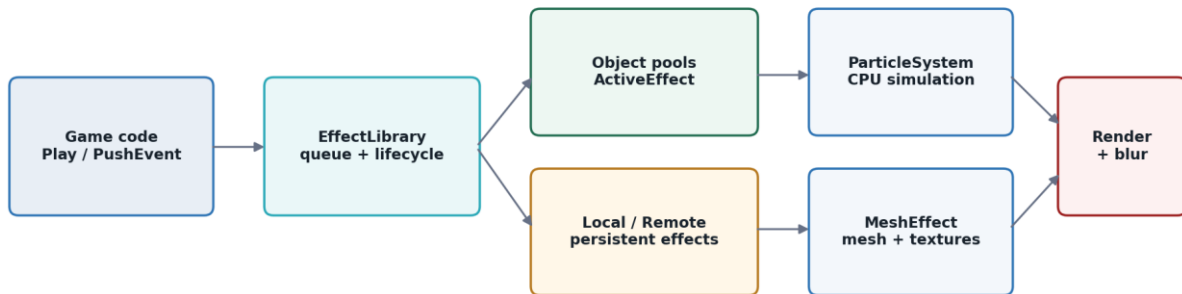


그림 1. 호출·수명 관리·풀·렌더링의 전체 구조

핵심 구성

구성 요소	책임
CEffectLibrary	초기화, 리소스·PSO·SRV 관리, 효과 풀, 이벤트 큐, 업데이트·렌더링, 포스트 프로세싱
CParticleSystem	CPU 입자 상태, 스폰·속도·중력·궤도 계산, GPU 업로드 버퍼 갱신, VS/GS/PS 기반 카메라 정면 빌보드 렌더링
CMeshEffect	반구형 메시 생성, DDS 3 중 텍스처, 월드 변환, 스크롤·색상, 인덱스 렌더링
ActiveEffect	실행 중 효과의 타입, 활성 상태, 나이·수명·루프, spread, 파티클/메시 포인터
EffectEvent	프레임 경계에서 소비하는 효과 요청: 타입·위치·크기·색·수명·루프

지원 효과 유형

그룹	EFFECT_TYPE	용도
기본	COLLISION, DUST	충돌·먼지 등 단발 입자
아이템	ITEM1 ~ ITEM11	리본·도형·BurstCore·ShockRing 텍스처 기반 효과
지속	BOOSTER, WIND_EFFECT	로컬/리모트 플레이어에 부착되는 지속 메시·입자
속도	SPEED_LINE	포스트 프로세스에서 화면 속도선 SRV 로 사용
궤도	LOCK_ORBIT, STUN_ORBIT	대상 위치를 따라 도는 궤도형 입자
드리프트	DRIFT_SPARK_LEFT, DRIFT_SPARK_RIGHT	좌우 바퀴 방향 스파크

2. 구성 파일과 프로젝트 설정

필수 파일

역할	파일
라이브러리	EffectLibrary.cpp / EffectLibrary.h
파티클	ParticleSystem.cpp / ParticleSystem.h
메시	MeshEffect.cpp / MeshEffect.h
공통	EffectPCH.cpp / EffectPCH.h
셰이더	EffectShaders.hlsl
DX12 도우미	d3dx12.h, DDSTextureLoader12.h 및 대응 구현 파일
에셋	Asset/DDS_File 아래 기본 DDS 텍스처

첨부 파일명 정리 EffectLibrary(53).cpp 같은 괄호 번호는 업로드 중 붙은 구분자입니다. 프로젝트에는 EffectLibrary.cpp, EffectLibrary.h 처럼 include 문과 일치하는 이름으로 추가합니다.

10.EffectShaders.hlsl 은 Visual Studio 에서 ‘빌드에서 제외: 예’로 설정합니다. 라이브러리가 실행 중 D3DCompileFromFile 로 셰이더를 컴파일합니다.

11.프로젝트 문자 집합·PCH 정책에 맞게 EffectPCH 를 조정하되,
DirectXMath·D3D12·D3DCompiler·DDS 로더 의존성은 유지합니다.

12.d3d12.lib, dxgi.lib, d3dcompiler.lib 등 기존 DX12 프로젝트의 링크 항목을 확인합니다.

에셋 경로

기본 텍스처는 실행 파일의 작업 디렉터리를 기준으로 상대 경로에서 로드됩니다. 폴더 구조를 바꾸려면 초기화 전에 SetEffectTextureFileName 또는 메시 텍스처 설정 함수를 호출합니다.

범주	기본 경로 예시
기본 입자	WhiteStar1.dds, Dust.dds
아이템	LongPinkRibbon.dds ~ YellowTriangle.dds, BurstCore.dds, ShockRing.dds
속도·궤도	SpeedLine1.dds, LockOrbit.dds, GreenStar.dds
드리프트	DriftSpark_Left.dds, DriftSpark_Right.dds
메시	noise.dds, BoosterBase.dds, BoosterNoise.dds, BoosterMask.dds

주의 EffectTypeConfig::textureFile 멤버만 변경해도 실제 SRV 로딩 경로는 바뀌지 않습니다. 기본 타입의 파일 교체는 SetEffectTextureFileName(type, path)를 초기화 전에 사용합니다. BOOSTER·WIND_EFFECT 는 이 경로 대신 메시 전용 API(SetBoosterMeshConfig 등)를 사용합니다 — 상세 내용은 부록 A 참고.

렌더 타겟·텍스 포맷 조건

현재 PSO 는 RTV_FORMAT=DXGI_FORMAT_R8G8B8A8_UNORM,
DSV_FORMAT=DXGI_FORMAT_D24_UNORM_S8_UINT, SampleCount=1 로 생성됩니다. 다른 포맷 또는
MSAA 를 쓰는 프로젝트는 BuildPipelineState 와 포스트 프로세스 텍스처 포맷을 프로젝트 설정과 맞춰야
합니다.

3. 초기화 및 종료

초기화 전 설정

폴 크기·입자 수·텍스처·메시 구조는 리소스 생성에 영향을 주므로 Initialize 보다 먼저 설정하는 것이
원칙입니다.

예시 1. 리소스 생성 전 경로·메시 설정

```
auto* effects = CEffectLibrary::Instance();

effects->SetEffectTextureFileName(EFFECT_TYPE::DUST, L"Assets/Effects/Dust.dds");

EffectMeshConfig booster;
booster.radius = 2.0f;
booster.sliceCount = 20;
booster.stackCount = 20;
booster.textureFiles = {
    L"Assets/Effects/BoosterBase.dds",
    L"Assets/Effects/BoosterNoise.dds",
    L"Assets/Effects/BoosterMask.dds"
};
effects->SetBoosterMeshConfig(booster);
```

렌더 리소스 초기화

예시 2. 디바이스·업로드 커맨드 리스트가 준비된 직후

```
auto* effects = CEffectLibrary::Instance();
effects->Initialize(m_pd3dDevice, m_pd3dCommandList);
effects->InitializePostProcess(
    m_pd3dDevice,
    m_nWndClientWidth,
    m_nWndClientHeight
);
```

- 13.ID3D12Device 와 기록 가능한 그래픽 커맨드 리스트를 준비합니다.
- 14.Initialize 를 호출해 루트 시그니처, PSO, SRV 힙, 텍스처와 효과 풀을 생성합니다.
- 15.업로드 명령을 실행하고 프로젝트의 일반적인 펜스 동기화 절차를 수행합니다.
- 16.화면 효과를 쓸 경우 InitializePostProcess 를 호출합니다.

분리된 초기화 Initialize 는 파티클·메시 렌더 리소스와 풀을 만들고, InitializePostProcess 는 SceneRenderTexture·BlurTexture·Compute PSO 를 만듭니다. 포스트 프로세싱을 쓰지 않는 프로젝트는 두 번째 호출을 생략할 수 있습니다.

해상도 변경

```
void OnResize(int width, int height)
{
    CEffectLibrary::Instance()->ResizePostProcess(
        m_pd3dDevice, width, height
    );
}
```

스왑 체인 크기 변경이 끝난 뒤 새 폭·높이로 호출합니다. 장치가 유효하고 포스트 프로세스 초기화가 완료된 상태여야 합니다.

종료

```
WaitForGpuIdle();
CEffectLibrary::Instance()->Release();
```

GPU 동기화 Release 는 텍스처·버퍼·PSO·힙을 즉시 해제합니다. 종료 또는 재초기화 전에 해당 리소스를 참조하는 GPU 작업이 끝났는지 확인합니다.

4. 프레임 업데이트와 렌더링 순서

포스트 프로세싱을 사용할 때는 라이브러리의 SceneRenderTexture 를 먼저 렌더 타겟으로 지정하고, 장면과 이펙트를 모두 그린 뒤 Compute 블러 결과를 BackBuffer 에 복사합니다.

예시 3. 매 프레임 권장 호출 순서

```
auto* effects = CEffectLibrary::Instance();

effects->Update(deltaTime);
effects->UpdateLocalBoosterPosition(playerPos, playerLook);
effects->UpdateRemoteBoosterPosition(remotePos, remoteLook);
effects->UpdateLockOrbitPosition(lockTargetPos);
effects->UpdateStunOrbitPosition (spinTargetPos);

effects->PrepareSceneRenderTarget(commandList, dsvHandle);

RenderScene(commandList);
RenderPlayers(commandList);
effects->Render(commandList, viewMatrix, projectionMatrix);

effects->SetPlayerSpeedRatio(currentSpeed / maxSpeed);
effects->RenderRadialBlur(
    commandList,
    currentBackBuffer,
    backBufferRtv,
    dsvHandle,
    static_cast<int>(currentSpeed)
);
```

단계	함수	핵심 역할
1	Update(dt)	이벤트 큐 소비, 입자·메시 갱신, 수명 종료 및 풀 반환, 속도선 상태 갱신
2	위치 동기화 API	플레이어·타겟이 움직인 뒤 지속/궤도 효과의 기준 위치 갱신
3	PrepareSceneRenderTarget	오프스크린 RT 로 전환하고 색·깊이 버퍼 초기화
4	프로젝트 Scene 렌더	맵, 오브젝트, 플레이어 등 본 장면 렌더링
5	Render(view, proj)	활성 파티클·메시 효과 렌더링
6	RenderRadialBlur	블러·속도선 Compute 처리 후 BackBuffer 로 복사
7	UI 렌더	블러 영향을 받지 않아야 하는 HUD 는 이 호출 뒤에 렌더링 권장

BackBuffer 상태 RenderRadialBlur 는 BackBuffer 가 PRESENT 상태라고 가정해 COPY_DEST 로 전환하고, 복사 뒤 RENDER_TARGET 상태로 둡니다. 프로젝트의 상태 추적 방식이 다르면 배리어 전후 상태를 맞춰 수정합니다.

포스트 프로세싱을 사용하지 않는 경우

```
effects->Update(deltaTime);
// 프로젝트의 BackBuffer/RT 가 이미 바인딩된 상태
RenderScene(commandList);
effects->Render(commandList, viewMatrix, projectionMatrix);
```

5. 효과 호출과 수명 제어

즉시 호출: Play

```
ActiveEffect* hitEffect = CEffectLibrary::Instance()->Play(
    EFFECT_TYPE::COLLISION,
    collisionPos,
    XMFLOAT2(18.0f, 18.0f),
    XMFLOAT3(1.0f, 0.75f, 0.3f)
);
```

파라미터	설명
type	EFFECT_TYPE 값. 해당 타입 풀이 비어 있으면 nullptr 반환
position	월드 기준 발생 위치
size	파티클 빌보드의 기준 가로·세로 또는 메시 스케일의 x·y
color	RGB 튜플. 원본색 유지 시 (1,1,1)
반환값	실행 인스턴스 ActiveEffect*. 위치 갱신·개별 중지에도 보관 가능

이벤트 큐 호출: PushEffectEvent

호출 시점과 실제 생성 시점을 분리하고 싶을 때 사용합니다. 요청은 큐에 들어가고 다음 Update 에서 FIFO 순서로 Play 됩니다.

```
effects->PushEffectEvent(
    EFFECT_TYPE::ITEM10,
    itemPos,
    XMFLOAT2(24.0f, 24.0f),
    XMFLOAT3(1.0f, 1.0f, 1.0f),
    0.18f,
    false
);
```

프레임 규칙 PushEffectEvent 만 호출하고 Update 를 생략하면 큐가 소비되지 않습니다. 게임 로직에서 이벤트를 넣더라도 매 프레임 Update 가 반드시 실행되어야 합니다.

이동하는 단발 효과

```
ActiveEffect* orbit = effects->Play(
    EFFECT_TYPE::LOCK_ORBIT, targetPos,
    XMFLOAT2(8.0f, 8.0f)
);

// 매 프레임
effects->UpdateEffectPosition(orbit, targetPos);

// 조기 종료
effects->StopEffect(orbit); // 포인터는 nullptr 로 정리됨
```

동일 타입의 모든 Lock Orbit 또는 Banana Spin 을 한 위치로 옮겨도 된다면 전용 UpdateLockOrbitPosition / UpdateStunOrbitPosition 을 사용할 수 있습니다. 개별 대상별 제어가 필요하면 반환 핸들과 UpdateEffectPosition 을 사용합니다.

타입 단위·전체 정리

함수	용도	운영 주의
StopEffect(ActiveEffect*&)	특정 인스턴스 조기 종료	핸들이 nullptr 로 바뀌므로 가장 안전한 개별 제어
StopEffectType(type)	같은 타입의 활성화 효과 모두 종료	BOOSTER/WIND_EFFECT 에는 사용하지 않는 것을

함수	용도	운영 주의
		권장
ClearActiveEffects()	활성 벡터 전체 제거	지속 부스터·바람막이까지 렌더 목록에서 빠지므로 일반 스테이지 전환용으로는 재초기화 정책 필요
GetActiveEffectCount()	활성 벡터 개수 확인	비활성 상태로 상주하는 지속 효과도 포함될 수 있음
GetPooledEffectCount(type)	남은 풀 개수 확인	0 이면 새 Play 가 nullptr 반환

6. ParticleConfig 와 사용자 정의 효과

ParticleConfig 주요 필드

범주	필드	의미
모션	motion	LINEAR / ORBIT / EMITTER
스폰	spawnShape	POINT / BOX / SPHERE / CIRCLE
방향·힘	direction, acceleration, gravity	초기 방향과 프레임 가속도
속도	minSpeed, maxSpeed	입자별 무작위 속도 범위
수명	minLifeTime, maxLifeTime	개별 입자 수명 범위
크기	startSize, endSize, shrink	기준 size 에 곱할 시작·끝 배율과 축소 사용 여부
색	startColor, endColor	수명 비율에 따른 색 보간 후 Play color와 곱함
축	moveX, moveY, moveZ	각 축의 위치 이동 허용
궤도	orbitRadius, orbitSpeed, orbitHeight	ORBIT 반경·각속도·높이
방출	emissionCount	EMITTER가 프레임마다 재활성화할 최대 입자 수

blendMode 현재 상태 ParticleBlendMode 필드는 설정 구조체에 포함되어 있지만, 현재 PSO 선택은 이 값을 직접 읽지 않습니다. 입자 PSO 는 기본적으로 SrcAlpha + One 가산 블렌딩입니다.

기본 타입 설정 변경

예시 4. Initialize 전에 풀·동작 설정

```
EffectTypeConfig cfg;
cfg.poolSize = 120;
cfg.particleCount = 6;
cfg.lifeTime = 0.45f;
cfg.spread = 16.0f;
cfg.loop = false;
cfg.useDepth = true;

cfg.particleConfig.motion = ParticleMotion::LINEAR;
cfg.particleConfig.spawnShape = ParticleSpawnShape::CIRCLE;
cfg.particleConfig.direction = XMFLOAT3(0.0f, 1.0f, 0.0f);
cfg.particleConfig.minSpeed = 20.0f;
cfg.particleConfig.maxSpeed = 55.0f;
cfg.particleConfig.startSize = XMFLOAT2(1.0f, 1.0f);
cfg.particleConfig.endSize = XMFLOAT2(0.15f, 0.15f);

effects->SetEffectTypeConfig(EFFECT_TYPE::COLLISION, cfg);
```

17.poolSize 와 particleCount 는 Initialize 시 실제 객체·버퍼 생성에 사용됩니다. 변경 후 반영하려면 초기화 전에 설정하거나 안전하게 재초기화합니다.

18.lifeTime, spread, loop, useDepth 와 particleConfig 는 이후 Play/Update/Render 동작에 반영됩니다.

19.SetEffectLifeTime 은 기본 타입의 수명만 간단히 바꿀 때 사용합니다.

HLSL 파티클 렌더링

실제 컴파일 엔트리는 VS_Particle → GS_Particle → PS_Particle 입니다. VS 는 월드 변환과 속성을 전달하고, GS 는 View 행렬의 right/up 축으로 한 점을 4 정점 카메라 정면 빌보드로 확장합니다. PS 는 텍스처 RGB 에 파티클 색을 곱하며 알파는 텍스처 알파를 사용합니다.

이름 기반 프리셋 등록

예시 5. 기존 타입 풀과 텍스처를 재사용하는 이름 기반 프리셋

```
EffectTypeConfig driftPreset = cfg;
driftPreset.lifeTime = 0.2f;
driftPreset.spread = 6.0f;

effects->RegisterEffect(
    "blue_drift_spark",
    EFFECT_TYPE::DRIFT_SPARK_LEFT,
    driftPreset
);

ActiveEffect* fx = effects->Play(
    "blue_drift_spark",
    wheelPos,
    XMFLOAT2(7.0f, 7.0f),
    XMFLOAT3(0.2f, 0.6f, 1.0f)
);
```

등록의 범위 RegisterEffect 는 새 EFFECT_TYPE·새 SRV·새 풀을 만드는 기능이 아닙니다. sourceType 의 기존 풀과 텍스처를 사용하면서 동작 설정을 이름으로 저장하는 프리셋입니다. sourceType 풀이 고갈되면 이름 기반 Play 도 nullptr 를 반환합니다.

7. 지속형 메시·부스터 제어

BOOSTER 와 WIND_EFFECT 는 초기화 시 로컬·리모트용 인스턴스를 각각 상주시킵니다. Toggle 함수가 활성 상태와 크기·색·오프셋을 바꾸고, Update 함수가 플레이어 위치와 Look 방향을 따라갑니다.

예시 6. 로컬 부스터

```
effects->ToggleLocalBooster(
    true,
    XMFLOAT3(2.0f, 10.0f, 2.0f),
    XMFLOAT3(0.1f, 0.5f, 1.0f),
    40.0f,
    50.0f
);

// 매 프레임 플레이어 이동 완료 후
effects->UpdateLocalBoosterPosition(playerPos, playerLook);

// 종료
effects->ToggleLocalBooster(false);
```

함수	대상
ToggleLocalBooster / UpdateLocalBoosterPosition	로컬 플레이어 부스터·바람막이
ToggleRemoteBooster / UpdateRemoteBoosterPosition	리모트 플레이어 부스터·바람막이
ToggleBooster / UpdateBoosterPosition	로컬 API 의 호환 별칭

20.lookDir 은 정규화되어 사용되며, 항상 0 이 아닌 벡터라는 전제로 설계되었습니다 — 상세 내용은 부록 A 참고.

21.boosterOffset 과 windOffset 은 현재 구현에서 모두 pos - look * offset 위치에 적용됩니다. 모델 전방축이 다른 프로젝트는 내부 헬퍼 함수(.cpp 파일 지역, 부록 A 참고)의 부호·회전을 직접 수정합니다.

22.부스터 PS_Booster 는 Base(t6)·Noise(t7)·Mask(t8)의 R 채널을 조합하고 gTintColor 를 적용합니다. 바람막이 PS_WindShield 는 스크롤 UV 와 가장자리 페이드를 적용하지만 현재 t6 만 샘플링하며 gTintColor 를 참조하지 않습니다. 따라서 SetColor 또는 m_WindMeshConfig.color 는 바람막이 화면색에 반영되지 않습니다.

메시 사전 설정

구분	기본값
바람막이	radius 15 / 20 slices / 20 stacks / scale (3,10.5,1.5) / scroll (0,-6,0)
부스터	radius 2 / 20 slices / 20 stacks / scale (2,10,2) / scroll (0,6,0)
텍스처 수	메시별 SRV 3 개 생성. 부스터는 t6~t8 모두 사용, 바람막이는 현재 t6 만 샘플링

8. 특수 효과 사용 예

차량 바퀴 먼지

```
effects->PlayCarDustParticle(
    EFFECT_TYPE::DUST,
    playerPos,
    playerRight,
    playerLook,
    XMFLOAT2(5.0f, 5.0f),
    XMFLOAT2(18.0f, 28.0f),
    XMFLOAT3(1.0f, 1.0f, 1.0f)
);
```

position 을 중심으로 right 방향 $\pm$ offset.x, look 방향 $\pm$ offset.y 조합의 네 지점에 Play 를 호출합니다. 호출한 번당 풀 인스턴스 4 개를 사용하므로 높은 빈도에서는 DUST 풀 잔량을 확인합니다.

드리프트 스파크

```
EFFECT_TYPE sparkType = isLeftWheel
    ? EFFECT_TYPE::DRIFT_SPARK_LEFT
    : EFFECT_TYPE::DRIFT_SPARK_RIGHT;

effects->Play(
    sparkType,
    wheelPos,
    XMFLOAT2(7.0f, 7.0f),
    XMFLOAT3(1.0f, 0.9f, 0.35f)
);
```

기본 설정은 좌우 각각 poolSize 100, particleCount 3, lifeTime 0.15 초, spread 8, depth 사용입니다.

Lock Orbit / Stun Spin

타입	기본 구성	권장 갱신
LOCK_ORBIT	pool 10 / particles 8 / 3.0 초 / ORBIT / shrink=false	UpdateEffectPosition(handle, targetPos)
STUN_ORBIT	pool 10 / particles 5 / 1.2 초 / ORBIT / shrink=false	UpdateEffectPosition(handle, playerPos)

두 타입은 Play 시 ResetLockOrbit 를 사용해 입자를 원 둘레에 균등 배치합니다. 궤도 반경·속도·높이는 해당 타입의 ParticleConfig 에서 조정합니다.

9. 포스트 프로세싱: 방사형 블러와 속도선

InitializePostProcess 는 SceneRenderTexture(SRV/RTV), BlurTexture(UAV), SpeedLine SRV 와 Compute PSO 를 준비합니다. CS_RadialBlur 는 $32 \times 32 \times 1$ 스레드 그룹과 픽셀 경계 검사를 사용해 화면 전체를 처리한 뒤 결과를 BackBuffer 에 복사합니다.

구현 주의: 첨부 C++의 Dispatch 는 $(width+15)/16$, $(height+15)/16$ 을 사용해 HLSL 의 32×32 그룹과 불일치합니다. 경계 검사 때문에 출력 오류는 없지만 불필요한 그룹이 실행되므로 $(width+31)/32$, $(height+31)/32$ 로 맞추는 것을 권장합니다.

항목	현재 구현
Compute shader	EffectShaders.hlsl / CS_RadialBlur / cs_5_1
루트 상수	CB_RADIAL_BLUR 8 개 32-bit 값
속도 기준	RenderRadialBlur 내부 maxSpeed = 300.0f
블러 강도	$\min(\text{speed} \times 0.0005, 0.15)$, speed <= 0 이면 0
속도선 임계	speedRatio > 0.7 에서 alpha 증가
중심·화면비	center (0.5, 0.5), aspect = width / height
스레드 그룹	[numthreads(32, 32, 1)] / Dispatch 권장: $\text{ceil}(\text{width} \div 32)$, $\text{ceil}(\text{height} \div 32)$
리소스 바인딩	b0 BlurParams / t0 Scene / t1 SpeedLine / u0 Output / s0 Sampler
블러 샘플	픽셀당 8 회, 중심 방향·화면비를 보정해 누적 후 속도선과 합성

속도 기준 일치 SetPlayerSpeedRatio 는 UpdateSpeedLineState 에도 사용되지만, RenderRadialBlur 는 전달된 speed 와 내부 maxSpeed=300 으로 sAlpha 를 다시 계산합니다. 게임 최고속도가 300 이 아니면 내부

기준값을 프로젝트 값과 맞추는 것이 좋습니다.

UI 를 선명하게 유지하는 순서

23.PrepareSceneRenderTarget

24.월드·플레이어·3D 이펙트 렌더

25.RenderRadialBlur

26.BackBuffer 에 HUD·텍스트·메뉴 렌더

10. API 빠른 참조

영역	API	요약
생명주기	Instance / Initialize / Release	싱글톤 접근, 렌더 리소스·폴 생성, 전체 해제
프레임	Update / Render	수명·모션 갱신, 활성 효과 렌더
즉시	Play(type, ...) / Play(name, ...)	폴에서 인스턴스를 꺼내 즉시 실행
지연	PushEffectEvent(...)	요청 큐 적재, 다음 Update 에서 소비
설정	SetEffectTypeConfig / SetEffectLifeTime	기본 타입 동작 변경
에셋	SetEffectTextureFileName	기본 타입 DDS 경로 변경; Initialize 전에 사용 (BOOSTER·WIND_EFFECT 제외 — 부록 A)
프리셋	RegisterEffect / UnregisterEffect / HasEffect / GetEffectConfig	sourceType 기반 이름 프리셋 관리
제어	UpdateEffectPosition / StopEffect / StopEffectType	개별 이동·중지, 타입 일괄 중지
상태	GetActiveEffectCount / GetPooledEffectCount	활성 벡터·남은 폴 수 확인
부스터	Toggle/Update Local·Remote Booster	지속형 플레이어 부착 효과
특수	PlayCarDustParticle / UpdateLockOrbitPosition / UpdateStunOrbitPosition	차량 4 점 먼지·동일 타입 일괄 위치 갱신

영역	API	요약
포스트	InitializePostProcess / ResizePostProcess / PrepareSceneRenderTarget / RenderRadialBlur	오프스크린 장면과 Compute 후처리

기본 풀·수명 설정

타입	풀	입자	수명	Depth	비고
COLLISION	50	3	2.0s	No	spread 20
DUST	2000	1	2.0s	Yes	gravity 9.8, moveZ=false
ITEM1~9	50	3	2.0s	No	spread 50
ITEM10	50	1	0.18s	No	spread 0
ITEM11	50	1	0.35s	No	spread 0
BOOSTER	상주+pool	50	loop	Yes	로컬·리모트 지속 인스턴스
WIND_EFFECT	상주	-	loop	-	로컬·리모트 메시
LOCK_ORBIT	10	8	3.0s	Yes	ORBIT
DRIFT L/R	100	3	0.15s	Yes	spread 8
STUN_ORBIT	10	5	1.2s	Yes	ORBIT

표의 미지정 항목은 EffectTypeConfig 또는 ParticleConfig 생성자의 기본값을 따릅니다.

11. 호환성 및 이식 가이드

검증 결과 본 라이브러리는 기존 게임을 비롯하여 Microsoft DirectX 12 샘플 프로젝트와 교수 제공 샘플 프로젝트에 적용했으며, 각 환경에서 정상적으로 연동·동작함을 확인했습니다..

호환성은 특정 GameFramework·Player 클래스에 직접 의존하지 않고, 표준

ID3D12Device·ID3D12GraphicsCommandList·DirectXMath 행렬·D3D12 핸들을 경계로 사용하기 때문에 확보됩니다. 다만 아래 렌더 계약은 대상 프로젝트와 일치해야 합니다.

검사 항목	라이브러리 기본값 / 요구사항	대상 프로젝트 대응
RTV 포맷	R8G8B8A8_UNORM	스왑 체인/오프스크린 포맷이 다르면 PSO와 텍스처 포맷 수정
DSV 포맷	D24_UNORM_S8_UINT	D32 등 사용 시 PSO DSVFormat 수정
샘플 수	1	MSAA 사용 시 SampleDesc 일치
셰이더	D3DCompileFromFile, Shader Model 5.1 / b6·b7, t6~t8, u0 고정 슬롯	EffectShaders.hlsl 작업 경로·엔트리 포인트와 Root Signature의 CBV/SRV/UAV 슬롯 일치 확인
텍스처	DDS + DDSTextureLoader12	상대 경로와 로더 구현 파일 포함
커맨드 리스트	그래픽 + Compute dispatch 가능	Direct command list 사용 권장
BackBuffer 상태	PRESENT → COPY_DEST → RENDER_TARGET	프로젝트 상태 추적과 충돌하지 않게 조정
행렬	view/proj를 내부에서 transpose	호출 측 행렬 규약을 중복 전치하지 않도록 확인
UI 순서	블러 후 RT 상태	후처리 후 HUD 렌더, 이후 PRESENT 배리어는 프레임워크가 수행

프로젝트별 이식 절차

27.파일·셰이더·DDS 로더를 프로젝트에 추가하고 include 경로와 링크 라이브러리를 확인합니다.

28.대상 프로젝트의 RTV/DSV/MSAA 포맷을 라이브러리 PSO와 맞춥니다.

- 29.에셋 경로와 실행 작업 디렉터리를 확인합니다.
- 30.디바이스·업로드 커맨드 리스트 생성 직후 Initialize 를 호출합니다.
- 31.후처리를 쓸 때 InitializePostProcess 및 리사이즈 훅을 연결합니다.
- 32.프레임 호출 순서를 적용하고 BackBuffer 상태 전이를 맞춥니다.
- 33.COLLISION 하나로 Play·Update·Render 를 확인한 뒤 부스터·블러 순으로 기능을 확장합니다.
- 34.종료 전 GPU idle 을 보장하고 Release 를 호출합니다.

12. 문제 해결

증상	확인할 원인	해결
Play 가 nullptr	타입이 잘못됐거나 해당 풀 고갈	GetPooledEffectCount 확인, 호출 빈도·poolSize·lifeTime 조정
텍스처가 안 보임	상대 경로·DDS 로드 실패·SRV 미생성 (BOOSTER/WIND_EFFECT 는 부록 A 참고)	OutputDebugString 의 Failed to load texture 확인, 작업 디렉터리 수정
셰이더/PSO 생성 실패	EffectShaders.hlsli 경로·엔트리·포맷 불일치	디버그 출력 확인, VS/PS/GS/CS 이름과 RTV/DSV 포맷 수정
이펙트가 벽 뒤에도 보임	해당 타입 useDepth=false	Initialize 전/재초기화 정책에 맞춰 useDepth 설정
부스터가 따라오지 않음	위치 갱신 누락·0 look 벡터	이동 완료 후 매 프레임 UpdateLocal/RemoteBoosterPosition 호출
Lock/Stun 궤도 효과가 한곳에 모임	타입 전체 위치 갱신 API 사용	각 Play반환 핸들에 UpdateEffectPosition 적용
블러 화면 오류	Resize 누락·리소스 상태·포맷 불일치	ResizePostProcess, 배리어 전후 상태, R8G8B8A8 포맷 점검
HUD 까지 흐려짐	UI 가 블러 전 Scene RT 에 렌더됨	RenderRadialBlur 뒤 BackBuffer 에 UI 렌더
재시작 후 부스터 미표시	ClearActiveEffects 가 지속 인스턴스를 제거	지속 타입 제외 또는 안전한 Release→Initialize 재구성

최종 통합 체크리스트

- 35.□ 필수 .cpp/.h, EffectShaders.hlsl, d3dx12.h, DDS 로더 구현 포함
- 36.□ EffectShaders.hlsl '빌드에서 제외' 설정
- 37.□ 기본 에셋 경로 또는 사용자 경로 확인
- 38.□ RTV·DSV·MSAA 설정 일치
- 39.□ Initialize·선택적 InitializePostProcess·ResizePostProcess 연결
- 40.□ Update → 위치 동기화 → Scene → Render → RenderRadialBlur 순서
- 41.□ BackBuffer 상태 전이 확인
- 42.□ 풀 고갈·nullptr 처리
- 43.□ GPU idle 후 Release

부록 A. 알려진 제약사항 및 향후 개선 과제

본 라이브러리는 졸업작품 범위에서 실동작을 검증했습니다. 아래 항목들은 현재 구현이 전제하고 있는 조건과, 이를 더 견고하게 만들기 위한 개선 방향을 정리한 것입니다.

항목	현재 구현	향후 개선 방향
BOOSTER/WIND_EFFECT 텍스처 경로	SetEffectTextureFileName 은 파티클 계열 타입에만 SRV 를 생성하며, LoadAssets 가 BOOSTER·WIND_EFFECT 인덱스는 건너뛴니다. 두 타입은 SetBoosterMeshConfig/SetBoosterText ureFiles, SetWindMeshConfig/SetWindTextureFil es 로 별도 관리됩니다.	두 경로를 하나의 설정 함수로 통합하거나, 잘못된 타입 호출 시 디버그 로그를 남기는 방어 코드 추가를 검토 중입니다.
부스터·바람막이 전방축	오프셋·회전	전방축 부호를 EffectMeshConfig 등 공개 설정

항목	현재 구현	향후 개선 방향
설정	계산(UpdateBoosterEffectSet)이 EffectLibrary.cpp 내부 static 헬퍼 함수로 고정되어 있어, 헤더를 통한 외부 설정이 불가능합니다.	구조체로 노출해 프로젝트별로 조정할 수 있도록 확장할 계획입니다.
lookDir 0 벡터 처리	UpdateLocal/RemoteBoosterPosition 은 lookDir 을 그대로 정규화하며, ParticleSystem::CreateVelocity 와 달리 0 벡터 방어 코드가 없습니다. 호출 측이 항상 유효한 방향 벡터를 전달한다는 전제로 설계했습니다.	XMVector3Normalize 이전에 길이 체크를 추가해, 0 벡터 입력 시 직전 방향을 유지하거나 기본값으로 대체하도록 보강할 예정입니다.
ClearActiveEffects와 지속 이펙트	ClearActiveEffects 는 부스터·바람막이 같은 상주 인스턴스도 활성 목록에서 제거하며, 이후 Toggle 함수만으로는 렌더링 목록에 복귀하지 않습니다(12 장 문제 해결 참고).	지속 타임을 별도 목록으로 분리하거나, ClearActiveEffects에 '지속 타임 제외' 옵션 파라미터를 추가하는 방향을 검토 중입니다.
CS_RadialBlur Dispatch 크기	HLSL 은 32×32 이지만 C++은 16 기준으로 그룹 수를 계산합니다. 경계 검사로 결과는 유지되나 작업량이 증가합니다.	C++ 그룹 수 계산을 $(width+31)/32$, $(height+31)/32$ 로 수정합니다.
바람막이 색·텍스처 사용	PS_WindShield 는 t6 만 샘플링하고 gTintColor 를 읽지 않습니다. 생성한 두 추가 SRV 와 SetColor 값은 현재 화면 결과에 쓰이지 않습니다.	t7·t8 또는 gTintColor 를 셰이더에 반영하거나, 미사용 설정·리소스를 정리합니다.
미사용 VSParticle 함수	파일 상단 VSParticle 은 존재하지만 PSO 가 컴파일하는 실제 엔트리는 VS_Particle 입니다.	혼동 방지를 위해 레거시 함수를 제거하거나 주석으로 용도를 명시합니다.

부록 B. 문서 기준 소스

파일명	반영 범위
EffectLibrary.cpp	초기화, 풀, 렌더, 이벤트, 부스터, 후처리, 이름 등록
EffectLibrary.h	공개 API·구조체·기본 경로
ParticleSystem.cpp	스폰·속도·모션·보간·GPU 기록
ParticleSystem.h	ParticleConfig·모션·형상·블렌드
MeshEffect.cpp	반구 메시·DDS·변환·렌더
MeshEffect.h	메시 공개 제어 API
EffectShaders.hlsl	파티클 빌보드, 바람막이·부스터, 방사형 블러·속도선 구현